Trading-holiday scraper

How it was built, every logical step it takes, what the live run produced and what it refuses to do

Live run: 2026-10-19 08:41:28 UTC · overall validation WARN

Rows: 75: NYSE 34, NASDAQ 12, OPRA 14, CME 15

Code: deliverables/taskA_holidays/scraper/ (Python 3.12), offline fixture tests

Automation: .github/workflows/trading-holidays.yml, weekly and on demand

Published to: Google Drive (CSVs uploaded by hand) and the GitHub repository

Irakli — L2 Support Engineer candidate · 18 October 2026

1. Summary

The task was to collect trading holidays and early closes for several US exchanges, normalise them into one schema with UTC times, validate them, and publish them to Google Drive automatically. I treated it as a small production job rather than a one-off script: every source was inspected before a parser was written, every response body is kept on disk, every adapter fails independently, and the validation step reports problems instead of quietly fixing them.

The latest run published 75 rows: NYSE 34 (2026–2028), NASDAQ 12 (2026), OPRA 14 (the 2025 schedule) and CME 15 (Thanksgiving and Christmas 2026 plus all of 2027, from Trading Hours files I downloaded by hand). Of the 27 validation checks, 25 passed, 2 warned and 0 failed, so the overall result is WARN.

The result is WARN rather than PASS, and that is the right outcome. Every pair of sources that covers the same year agrees: NYSE and NASDAQ on every 2026 date, NYSE and CME on every 2027 date. CME rows come from Trading Hours Holidays Excel files I downloaded into input/cme/ (Equities row only); the site itself is still not scraped. The warnings are year coverage: NASDAQ Trader publishes the current year only, so 2027 is not online yet, and OPRA still serves only the 2025 PDF (2026 returned 403).

2. The files on Google Drive

These CSVs are the deliverable. The uploader finds each file by name and overwrites it, so a weekly run never creates duplicates. That is why the names must not change.

File	Contents	Rows	Size
trading_holidays_combined.csv	Every exchange in one table	75	20,387 B
trading_holidays_NYSE.csv	NYSE 2026–2028	34	9,092 B
trading_holidays_NASDAQ.csv	NASDAQ 2026 (only year published; no 2027 yet)	12	2,864 B
trading_holidays_OPRA.csv	OPRA, from the 2025 schedule PDF (no 2026/2027 PDF)	14	4,579 B
trading_holidays_CME.csv	CME Globex Equities from manual Trading Hours xlsx	15	4,311 B

Not uploaded: _previous_combined.csv (the baseline for the run-to-run diff), run_meta.json and upload_manifest.json (internal bookkeeping), and raw/ (evidence, not the calendar). When the Drive secrets are configured the job also builds one Google Sheet, trading_holidays, with a tab per exchange and a validation tab. The live run was a dry run; I uploaded the files above to Drive by hand, so the Drive API path itself is still unverified.

Column	Meaning
exchange	NYSE, NASDAQ, OPRA or CME
date	Session date, ISO format
holiday_name / canonical_name	Name as published / normalised name used for cross-exchange comparison
status	closed or early_close
close_time_local, timezone, close_time_utc	Early-close time in the exchange's zone and converted to UTC with zoneinfo (DST-correct)
session_or_product_scope	Which session or product the row applies to
source_url, scraped_at_utc, source_year	Provenance: where, when, and which year's publication

3. How the scraper works

There is one entry point, python -m scraper.run. It stamps a single UTC timestamp for the whole run (so a slow adapter cannot make one file look like two runs), calls the adapters in order, normalises the rows, validates them, writes the files and, if asked, uploads. Every fetched body is kept under raw/, and a crash in one adapter never stops the others. The steps below follow that order.

Step 1: Reconnaissance before any parser

Each candidate source was opened once and saved under raw/_recon_2026-10-16/ before I trusted it. Two of the five obvious URLs turned out to be the wrong choice, which is exactly why this step exists.

Source	What I found	Decision
nyse.com hours-calendars	Static HTML table for 2026–2028; early closes only in footnotes	Scrape the HTML
nasdaq.com holiday page	Non-browser clients time out or get an HTTP/2 reset	Fallback only (Playwright)
nasdaqtrader.com Calendar	Official operations page, same 2026 table over plain HTTPS	Primary NASDAQ source
opraplan.com	Yearly holiday PDF linked from the homepage (theocc.com is the clearing house and returns 403)	Follow the PDF links
cmegroup.com	HTTP 403 even for robots.txt; Data Terms of Use prohibit scripts	Do not scrape; read operator files

Step 2: Fetch politely, keep everything

All adapters share one fetch method. It sends normal browser headers (to identify the client, not to evade anything), saves every 200 response under raw/<exchange>/<run>/, and treats status codes differently on purpose:

- **401, 403, 404:** saved with an .httpNNN suffix and raised immediately and never retried. A site that has said no is not asked again.
- **429 and 5xx:** wait for Retry-After if given, otherwise 2^attempt plus jitter, then retry.
- **Network errors:** retried the same way, except for NASDAQ, where a connection reset means 'this client is refused'; it gets two quick tries and moves to the next source.

Step 3: Parse each exchange

NYSE

The page has one table: holiday names down the side and one column per year. A cell with a date becomes a closed row for that year; a cell with a dash means 'not observed' and produces a note, not an invented date: New Year's Day 2028 falls on a Saturday and NYSE publishes a dash, so there is no row. Early closes are not in the table at all; they live in the footnotes ('...will close early at 1:00 p.m. on...'). The adapter keeps the footnote text, finds the time and the weekday dates in it, and turns each into an early_close row. Live result: 29 closes and 5 early closes across 2026–2028.

NASDAQ

nasdaqtrader.com is tried first, with short timeouts. Its table is 'year | Holiday | Status'; a status containing a time is an early close at that local time. If it fails, headless Chromium opens the public nasdaq.com page and the rendered HTML is parsed instead. No stealth plugins, no proxies. If both fail, NASDAQ ships zero rows with a SOURCE_UNAVAILABLE note and the other exchanges still publish. Live result: nasdaqtrader.com answered with 12 rows; the Playwright fallback is tested on a saved page but was not needed live.

OPRA

OPRA publishes one PDF per year, linked from its homepage. The adapter collects links whose text says 'holiday schedule' with a year from last year onwards, downloads them a second apart, and reads them with pdfplumber: 'HOLIDAY NAME Month D' lines are closes, and the asterisked footnote lists the 13:00 ET early closes. On 16 October only the 2025 PDF was linked (the 2026 URL returned 403), so OPRA ships 14 rows from 2025, including 1 January 2026 printed on the 2025 notice.

CME: no scrape, parse operator downloads

There is no single CME holiday calendar: equity index, rates, energy and agriculture differ, and Globex differs from ClearPort. The adapter is scoped to Globex US equity index products (ES, NQ, YM, RTY) and never hits cmegroup.com from the job. It reads files dropped into input/cme/: Trading Hours → Holidays Excel exports (Equities row only), a documented CSV, or per-holiday PDFs. Empty cells on the holiday date become closed; a CLOSED before 16:00 CT becomes early_close; a day that only pre-opens the next trade date is treated as closed. Scraping a site whose terms forbid it is not a trade-off I would make for a client-facing job.

Year coverage gaps (NASDAQ and OPRA)

NYSE already publishes 2026–2028 on one page, so a refresh picks up 2027 without extra work. NASDAQ Trader's calendar table is current-year only; as of the last run that is 2026, and there is no honest way to invent 2027 rows until they publish them. OPRA still links the 2025 holiday PDF; probes of the 2026 URL returned 403, and 2027 is not posted. Those gaps stay WARN/documented, not filled by guesswork.

Step 4: Normalise

- Clean names (collapse whitespace, strip footnote marks) and map them to canonical names, so 'JUNTEENTH' in the OPRA PDF still matches 'Juneteenth' elsewhere.
- Convert early-close times to UTC using the exchange's IANA time zone: 13:00 New York on 24 December 2026 becomes 18:00Z (EST), on 3 July 2028 it becomes 17:00Z (EDT).
- Sort and drop duplicate (exchange, date, status) keys. There were none on this run.
- Trust the published date. An observed-date helper exists and is tested, but it is never used to invent a date the source did not print.

Step 5: Validate, without auto-correcting

Validation walks the table once and grades every check PASS, WARN or FAIL. The overall result is the worst grade. Differences between exchanges are reported, never edited away.

The cross-exchange checks are the ones that matter most. NYSE and NASDAQ list the same ten closures and the same two early closes in 2026. For 2027, where the CME files cover a full year, NYSE and CME equity index agree on all ten closures and on the early close the day after Thanksgiving. OPRA cannot be compared yet because its only schedule is 2025, which is why those two checks are WARN.

Check	Result	Detail
NYSE.rows	PASS	NYSE returned 34 rows
NASDAQ.rows	PASS	NASDAQ returned 12 rows
OPRA.rows	PASS	OPRA returned 14 rows
CME.rows	PASS	CME returned 15 rows
weekday	PASS	0 weekend-dated rows
duplicates	PASS	0 duplicate (exchange,date,status) rows
CME.2026.closures	PASS	2 full-day closures (plausible 2-14)
CME.2027.closures	PASS	10 full-day closures (plausible 2-14)
NASDAQ.2026.closures	PASS	10 full-day closures (plausible 8-11)
NYSE.2026.closures	PASS	10 full-day closures (plausible 8-11)
NYSE.2027.closures	PASS	10 full-day closures (plausible 8-11)
NYSE.2028.closures	PASS	9 full-day closures (plausible 8-11)
OPRA.2025.closures	PASS	10 full-day closures (plausible 8-11)
OPRA.2026.closures	PASS	1 closure(s) spilling from source_year 2025
early_close_time	PASS	0 early-close rows without a close time
NASDAQ.2026.early_closes	PASS	2 early closes (minimum expected 1)
NYSE.2026.early_closes	PASS	2 early closes (minimum expected 1)
NYSE.2027.early_closes	PASS	1 early closes (minimum expected 1)
NYSE.2028.early_closes	PASS	2 early closes (minimum expected 1)
NYSE_vs_NASDAQ.closed	PASS	years [2026]: 0 differing dates
NYSE_vs_NASDAQ.early_close	PASS	years [2026]: 0 differing dates
NYSE_vs_OPRA	WARN	no overlapping source years ([2026, 2027, 2028] vs [2025]); comparison skipped
NASDAQ_vs_OPRA	WARN	no overlapping source years ([2026] vs [2025]); comparison skipped
NYSE_vs_CME.2027.closed	PASS	0 differing dates
NYSE_vs_CME.2027.early_close	PASS	0 differing dates
good_friday	PASS	6 Good Friday rows checked against Easter-2
diff_previous	PASS	rows 75 -> 75 (0.0% change, threshold 30.0%); added 0: []; removed 0: []

New check. After an external review I added one more check: every full NYSE and NASDAQ year must contain at least one early close. Footnote parsing is the fragile part of the NYSE adapter, and a layout change that silently yields zero early closes would otherwise pass. Applied to the saved run it gives:

Check	Result	Detail
NASDAQ.2026.early_closes	PASS	2 early close(s); minimum 1
NYSE.2026.early_closes	PASS	2 early close(s); minimum 1
NYSE.2027.early_closes	PASS	1 early close(s); minimum 1
NYSE.2028.early_closes	PASS	2 early close(s); minimum 1

Step 6: Write the files and exit with a meaningful code

The combined CSV and one CSV per exchange that returned rows are written, plus validation_report.md, validation_checks.json and run_meta.json. Before overwriting, the previous combined file is copied aside so the next run can diff against it; a change of more than 30 % of rows is a FAIL. Exit code 0 means PASS or WARN, 1 means validation FAIL, 2 means an adapter crashed, and the automation treats those differently.

4. Automation on GitHub

The workflow lives at .github/workflows/trading-holidays.yml in the repository root, the only place GitHub executes workflows from.

1. **Triggers.** Every Monday at 06:17 UTC, plus a manual run with an exchange list and an upload switch.
2. **Setup.** Python 3.12, requirements.txt and Playwright's Chromium for the NASDAQ fallback.
3. **Tests first.** python -m pytest tests -q runs the offline fixture suite, no network.
4. **Run with retries that make sense.** Up to three attempts with backoff for a crash (exit 2); a validation FAIL (exit 1) stops immediately because retrying will not fix a bad calendar.
5. **Report.** The validation report goes into the run summary; output/ and raw/ are kept as a 90-day artifact.
6. **Publish.** With the Drive secrets set the files are uploaded; without them the job runs in DRY_RUN. The refreshed CSVs are committed back to the repository, skipping missing files and empty commits.
7. **Alert.** On failure, a Slack message with the run link and the FAIL/WARN lines, if SLACK_WEBHOOK_URL is set.

Secret	Purpose	Required
GOOGLE_SERVICE_ACCOUNT_JSON	Service-account key content for the Drive API	For upload
GDRIVE_FOLDER_ID	Target folder; empty means DRY_RUN	For upload
SLACK_WEBHOOK_URL	Failure alerts	Optional
GDRIVE_SHARE_WITH (variable)	Comma-separated readers to share with	Optional

What 'fetch everything' can and cannot mean. The job fetches NYSE, NASDAQ and every OPRA PDF linked from opraplan.com. It does not fetch CME and it does not invent an OPRA 2026 calendar. CME rows appear when the downloaded files are in input/cme/; those files are not in the repository, so the scheduled run on GitHub publishes no CME rows unless they are provided to it.

5. What was not done, and why

Item	Status	Why
Drive upload through the API	Partial	No service account in this environment; files uploaded by hand, API path unverified
CME automated scrape	Not done	403 + Data Terms of Use; rows come from manual Trading Hours xlsx instead
NASDAQ 2027	Not published	nasdaqtrader.com calendar is current year only; no 2027 table to fetch
OPRA 2026 / 2027	Blocked	2026 PDF URL returned 403; 2027 not posted on opraplan.com
nasdaq.com via Playwright, live	Partial	Tested on a saved page; not needed live
Cross-check with a third-party holiday library	Not done	Considered; the NYSE–NASDAQ cross-check already covers 2026

6. Adding another exchange

1. Create scraper/adapters/<name>.py with the exchange, time zone, source URL and scope, implementing scrape() and parse().
2. Use self.fetch so raw bodies are kept, and self.row(...) so the schema stays aligned.
3. Register it in scraper/adapters/__init__.py and give it a plausible closure band in validate.PLAUSIBLE_CLOSURES.
4. Save a fixture under tests/fixtures/ and add an offline parser test.
5. Add the name to the workflow's default exchange list.